COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	07/24/26	**DATE SUBMITTED:**	07/29/26
NAME:	Prince Benjamin S. Villaruel	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 Linked Lists

1. PROCEDURE OUTPUT

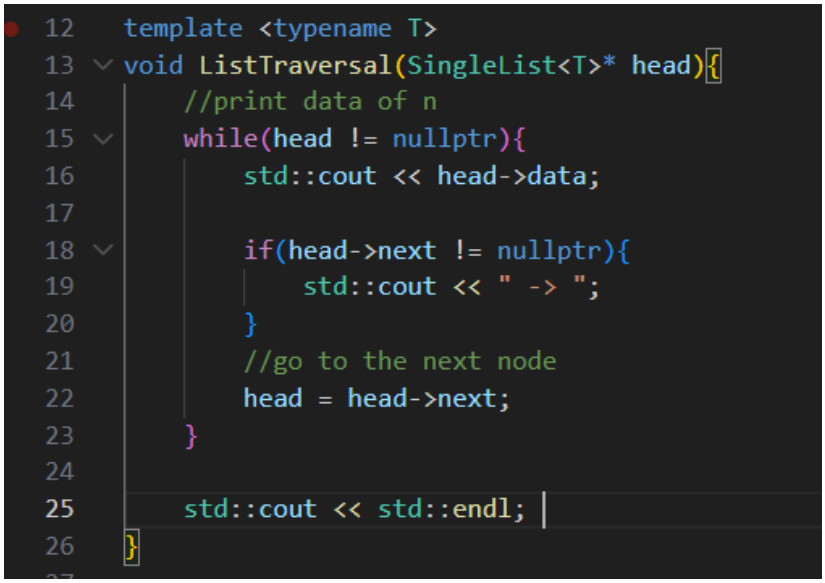
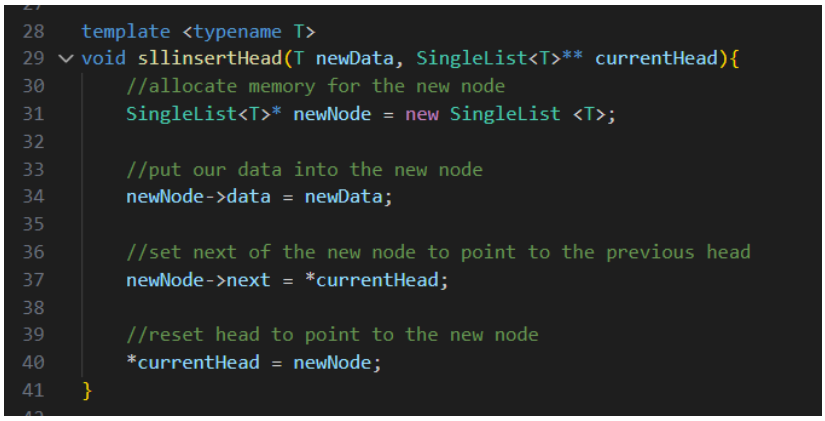
What procedures did you perform?

Table 3-1. Output of Initial/Simple Implementation

Screenshot	

Discussion	This implementation manually creates and links each node by hand, which shows what linked list looks like in memory but isn't practical, since a real implementation needs functions like insert and traverse to build the list dynamically.

Table 3-2. Code for the List Operations

Discussion	**Screenshot**
Traversal	
Insertion at head	

```
12  template <typename T>
13  void ListTraversal(SingleList<T>* head){
14      //print data of n
15      while(head != nullptr){
16          std::cout << head->data;
17
18          if(head->next != nullptr){
19              std::cout << " -> ";
20          }
21          //go to the next node
22          head = head->next;
23      }
24
25      std::cout << std::endl;
26  }
```

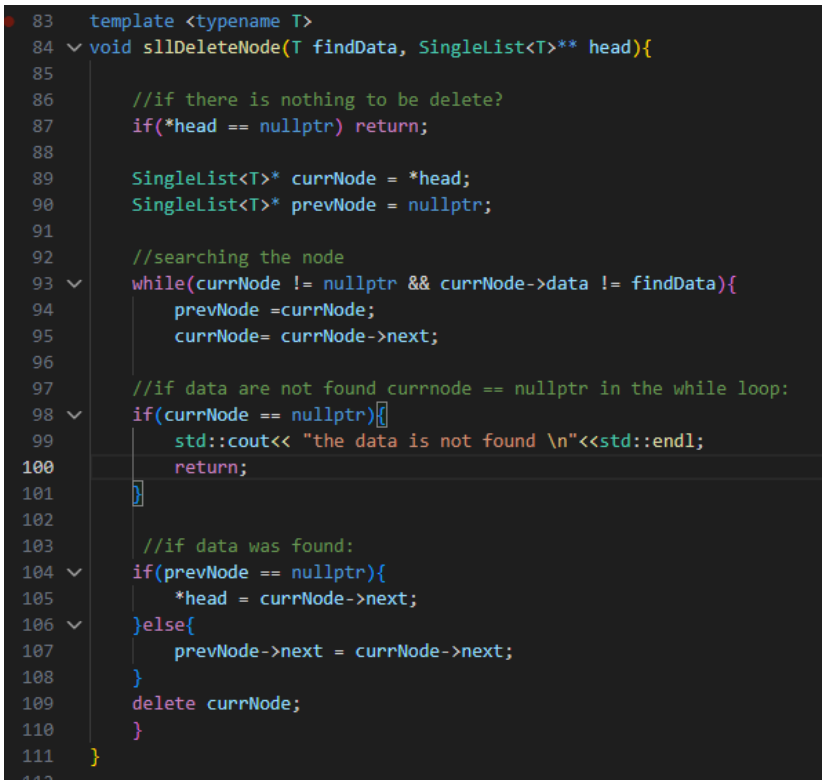
```
28  template <typename T>
29  void sllinsertHead(T newData, SingleList<T>** currentHead){
30      //allocate memory for the new node
31      SingleList<T>* newNode = new SingleList <T>;
32
33      //put our data into the new node
34      newNode->data = newData;
35
36      //set next of the new node to point to the previous head
37      newNode->next = *currentHead;
38
39      //reset head to point to the new node
40      *currentHead = newNode;
41  }
```

Insertion at any part of the list

```
43  template <typename T>
44  void sllGeneralInsert(T newData, SingleList<T>* preNode){
45      //check if it is the head node(previous node is null)
46      if(preNode ==nullptr){
47          //if null, print "previous node cannot be null"
48          std::cout<<"Previous value cannot be null"<<std::endl;
49          return;
50      }
51      //allocate new node
52      SingleList<T>* newNode = new SingleList<T>;
53
54      //store data in the new node
55      newNode->data = newData;
56
57      //point new node to the previous node
58      newNode->next = preNode->next;
59
60      //point previous
61      preNode->next = newNode;
62
63  }
```

Insertion at the end

```
65   template <typename T>
66   void sllinsertEnd(T newData, SingleList<T>** head){
67       //allocate a new node
68       SingleList<T>* newNode = new SingleList<T>;
69       //dereference to the new node
70       SingleList<T>* currentNode = *head;
71       //store data in new node
72       newNode-> data = newData;
73       //point to null
74       newNode->next = nullptr;
75       //traverse the list until next of the node is null
76       while(currentNode->next != nullptr){
77           currentNode = currentNode->next;
78       }
79       //point to the next of the current node to the new node
80       currentNode->next = newNode;
81   }
```

Deletion of a node	

```
83  template <typename T>
84  void sllDeleteNode(T findData, SingleList<T>** head){
85
86      //if there is nothing to be delete?
87      if(*head == nullptr) return;
88
89      SingleList<T>* currNode = *head;
90      SingleList<T>* prevNode = nullptr;
91
92      //searching the node
93      while(currNode != nullptr && currNode->data != findData){
94          prevNode =currNode;
95          currNode= currNode->next;
96
97      //if data are not found currnode == nullptr in the while loop:
98      if(currNode == nullptr){
99          std::cout<< "the data is not found \n"<<std::endl;
100         return;
101     }
102
103      //if data was found:
104     if(prevNode == nullptr){
105         *head = currNode->next;
106     }else{
107         prevNode->next = currNode->next;
108     }
109     delete currNode;
110     }
111 }
```

Table 3-3. Code and Analysis for Singly Linked Lists

a. Source code

newmain.cpp > main()

```cpp
#include <iostream>
#include "singly_ll.h"

int main(){
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;
    SingleList<char>* fifth = nullptr;
    SingleList<char>* sixth = nullptr;

    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;
    fifth = new SingleList<char>;
    sixth = new SingleList<char>;

    head->data = 'C';
    second->data = 'P';
    third->data = 'E';
    fourth->data = '1';
    fifth->data = '0';
    sixth->data = '1';

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;
    fifth->next = sixth;
    sixth->next = nullptr;

    std::cout<< "Testing of traversal: ";
    ListTraversal(head);
```

Console

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Linked Lists>
Testing of traversal: C -> P -> E -> 1 -> 0 -> 1
```

b. Source code

```cpp
    std::cout<<"Testing of sllinsertHead: \n";
    sllinsertHead('G', &head);
    ListTraversal(head);
```

Console

```
Testing of sllinsertHead:
G -> C -> P -> E -> 1 -> 0 -> 1
```

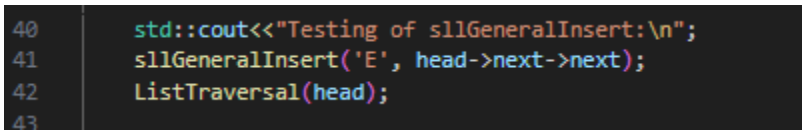
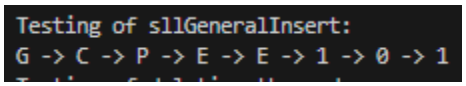
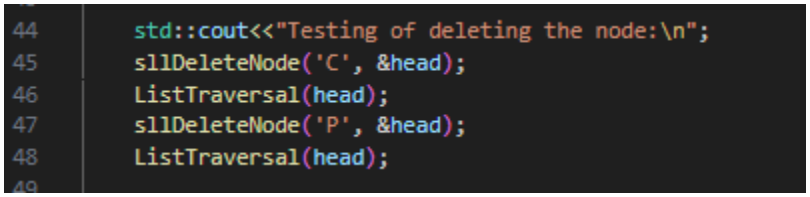
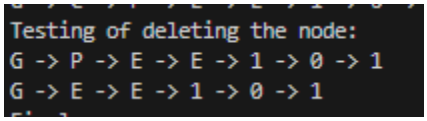
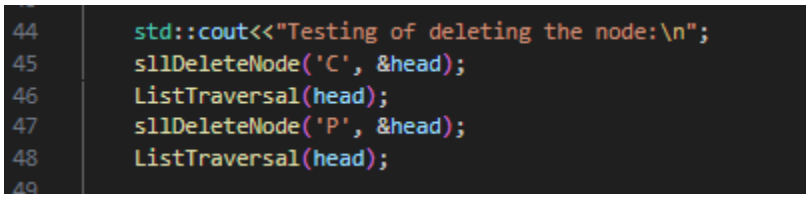
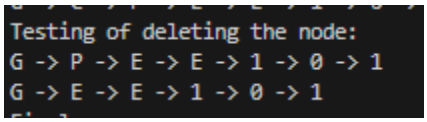
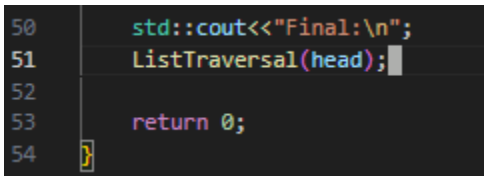
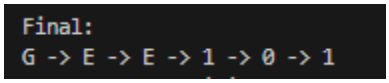
c.	Source code	
	Console	
d.	Source code	
	Console	
e.	Source code	
	Console	
f.	Source code	
	Console	

Table 3-4. Modified Operations for Doubly Linked Lists

Screenshots	Analysis



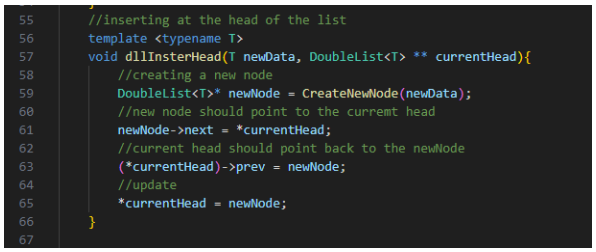
```cpp
template <typename T>
void dllTraverse(DoubleList<T>* currentNode){
    //temp variable to store the last node visited
    DoubleList<T> *tail;

    //check if the node is empty
    if(currentNode == nullptr){
        std::cout<<"The list is empty."<<std::endl;
        return;
    }
    std::cout<<"Forward \n";
    //move forward
    while(currentNode != nullptr){
        std::cout<< currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode-> next;

    }
    //add a next line
    std::cout<<std::endl;
    //move backward
    std::cout<<" \n";

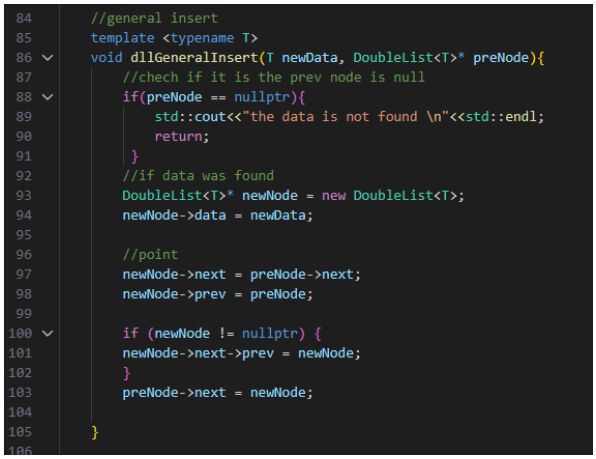
    while(tail != nullptr){
        std::cout<< tail->data<<" ";
        tail = tail->prev;
    }
}
```

This function traverses forward while saving the last node visited as tail, then uses that same tail to traverse backward with prev, showing how the prev pointer lets a double linked list move both directions in one pass.



```cpp
//inserting at the head of the list
template <typename T>
void dllInsterHead(T newData, DoubleList<T> ** currentHead){
    //creating a new node
    DoubleList<T>* newNode = CreateNewNode(newData);
    //new node should point to the curremt head
    newNode->next = *currentHead;
    //current head should point back to the newNode
    (*currentHead)->prev = newNode;
    //update
    *currentHead = newNode;
}
```

This function inserts at the head by pointing newNode->next to the old head and the old head's prev back to the newNode, then updates *currentHead to the new node.



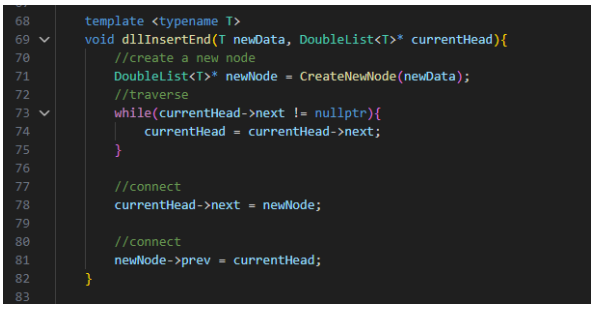
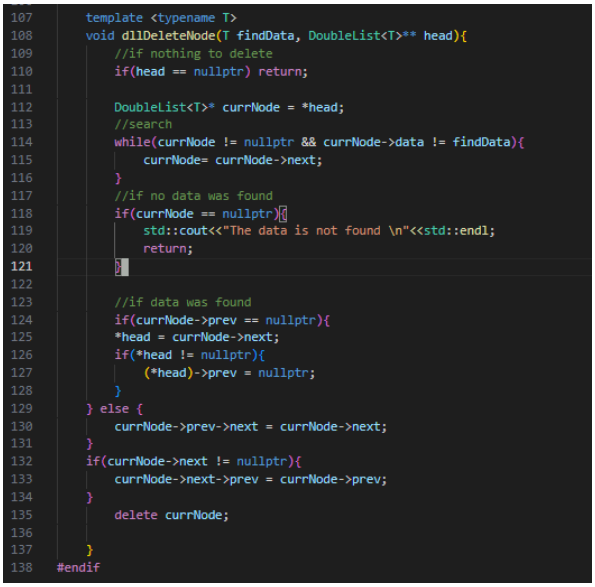
```cpp
//general insert
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* preNode){
    //chech if it is the prev node is null
    if(preNode == nullptr){
        std::cout<<"the data is not found \n"<<std::endl;
        return;
    }
    //if data was found
    DoubleList<T>* newNode = new DoubleList<T>;
    newNode->data = newData;

    //point
    newNode->next = preNode->next;
    newNode->prev = preNode;

    if (newNode != nullptr) {
    newNode->next->prev = newNode;
    }
    preNode->next = newNode;

}
```

This function inserts a new node after preNode by linking newNode-> next to preNode->prev back to preNode, updating the old next node's prev if it exists, then finally setting preNode->next to newNode.

	This function inserts at the end by traversing forward until currentHead->next is nullptr, then links the new node with currentHead->next = newNode and newNode ->prev = currentHead, attaching it as the new tail.
	This function searches for the node with findData, then removes it by relinking its neighbors. If it's the head, *head is updated to the next node; otherwise the previous node's next and the next node's prev are re-pointed around it, before the node is deleted.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Supplementary Activity



scmain.cpp

```cpp
#include <iostream>
#include <string>
#include "soundcloud_ll.h"

int main(){
    //create queue
    CircularList<std::string>* queue = nullptr;
    addSong(std::string("ALL NIGHT LONG"), &queue);
    addSong(std::string("AYUSING ANG SIRA"), &queue);
    addSong(std::string("LIL SHAWTY"), &queue);
    addSong(std::string("FADED OFF"), &queue);

    std::cout << "Queue created. Playing all songs:\n";
    playAllSongs(queue);

    //add song
    std::cout << "\n Adding 'AWAY' to the queue:\n";
    addSong(std::string("AWAY"), &queue);
    playAllSongs(queue);

    //remove song
    std::cout << "\n Removing 'LIL SHAWTY':\n";
    removeSong(std::string("LIL SHAWTY"), &queue);
    playAllSongs(queue);

    //loop
    std::cout << "\n Playing all songs (loop):\n";
    playAllSongs(queue);

    //testing
    CircularList<std::string>* nowPlaying = queue;
    std::cout << "\n Now playing: " << nowPlaying->data << std::endl;
    nextSong(&nowPlaying);
    std::cout << " next -> " << nowPlaying->data << std::endl;
    nextSong(&nowPlaying);
    std::cout << " next -> " << nowPlaying->data << std::endl;
    nextSong(&nowPlaying);
    std::cout << " previous -> " << nowPlaying->data << std::endl;

    //track that does not exist
    std::cout << "\n Removing a track that does not exist ('Paano ka ba?'):\n";
    removeSong(std::string("Paano ka ba?"), &queue);

    //wraparound
    std::cout << "\n Wraparound:\n";
    CircularList<std::string>* walker = queue;
    for(int i = 0; i < 6; i++){
        std::cout << " " << walker->data << std::endl;
        nextSong(&walker);
    }


    return 0;
}
```

scmain.cpp | soundcloud_ll.h

soundcloud_ll.h > ...

```cpp
#ifndef SOUNDCLOUD_LL_H
#define SOUNDCLOUD_LL_H

#include <iostream>

//node creation
template <typename S>
class CircularList{
    public:
    S data;
    CircularList<S>* next = nullptr;
    CircularList<S>* prev = nullptr;
};
//add song at the end of our SC queue
template <typename S>
void addSong(S newSong, CircularList<S>** head){
    CircularList<S>* newNode = new CircularList<S>;
    newNode->data = newSong;

    //empty queue
    if(*head == nullptr){
        newNode->next = newNode;
        newNode->prev = newNode;
        *head = newNode;
        return;
    }
    CircularList<S>* tail = (*head)->prev;

    //linking
    tail->next = newNode;
    newNode->prev = tail;
    newNode->next = *head;
    (*head)->prev = newNode;
}

//remove a song
template <typename S>
void removeSong(S findSong, CircularList<S>** head){
    if(*head == nullptr) return;

    CircularList<S>* currNode = *head;

    do{
        if(currNode->data == findSong){
            if(currNode->next == currNode){
                *head = nullptr;
            }else{
                currNode->prev->next = currNode->next;
                currNode->next->prev = currNode->prev;

                if(currNode == *head){
                    *head = currNode->next;
                }
            }
            delete currNode;
            return;
        }
        currNode = currNode->next;
    }while(currNode != *head);

    std::cout << "\"" << findSong << "\" was not found in the queue\n";
}
```

```cpp
//play every song once
template <typename S>
void playAllSongs(CircularList<S>* head){
    if(head == nullptr){
        std::cout << "the queue was empty\n";
        return;
    }
    CircularList<S>* currNode = head;
    do {
        std::cout << currNode->data;
        currNode = currNode->next;
        if(currNode != head){
            std::cout << " -> ";
        }
    }while(currNode != head);

    std::cout << std::endl;
}

template <typename S>
void nextSong(CircularList<S>** current){
    if(*current == nullptr) return;
    *current = (*current)->next;
}

template <typename S>
void previousSong(CircularList<S>** current){
    if(*current == nullptr) return;
    *current = (*current)->prev;
}


#endif
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Linked Lists> cd
Queue created. Playing all songs:
ALL NIGHT LONG -> AYUSING ANG SIRA -> LIL SHAWTY -> FADED OFF

 Adding 'AWAY' to the queue:
ALL NIGHT LONG -> AYUSING ANG SIRA -> LIL SHAWTY -> FADED OFF -> AWAY

 Removing 'LIL SHAWTY':
ALL NIGHT LONG -> AYUSING ANG SIRA -> FADED OFF -> AWAY

 Playing all songs (loop):
ALL NIGHT LONG -> AYUSING ANG SIRA -> FADED OFF -> AWAY

 Now playing: ALL NIGHT LONG
 next -> AYUSING ANG SIRA
 next -> FADED OFF
 previous -> AWAY

 Removing a track that does not exist ('Paano ka ba?'):
"Paano ka ba?" was not found in the queue

 Wraparound:
 ALL NIGHT LONG
 AYUSING ANG SIRA
 FADED OFF
 AWAY
 ALL NIGHT LONG
 AYUSING ANG SIRA
```

Observations

What stood out the most is that a circular linked list isn't just a variation for practice, it's a natural fit for something like a repeating music queue. The structure worked because the tail's next always pointed back to the head, and the head's prev always pointed back to the tail, so the list never had a dead end.

Conclusion

Working through single, double, and circular linked lists helped me see how each one builds on the last. The single linked list taught me the basics of dynamic node creation and traversal using just a next pointer, but I quickly noticed its limitation of only being able to move forward. Adding a previous pointer in the double linked list let me move backward just as easily, which made operations like removal simpler since I didn't need to keep track of a separate previous node while traversing. Finally, turning that structure into a circular linked list, by linking the tail back to the head, showed me how a list can loop indefinitely without ever hitting a nullptr, which was essential for something like a continuously playing music queue. Overall, this was the lesson I understood the most, and it made it this activity the most fun I've ever had, giving me the confidence I'll carry into future laboratories.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.